Homework 2
Distributed Systems

Team 61

Saharsh Kallu (2024101129)
saharsh.kallu@students.iiit.ac.in

Aditya Rathore (2025801016)
aditya.rathore@research.iiit.ac.in

September 4, 2026

1 Experimental Setup

All experiments were run on one compute node (`node07`) with up to eight MPI processes. The node uses an Intel Xeon Gold 5317 CPU at 3.00 GHz. The code was compiled with GCC 8.5.0 and Open MPI 4.0.4rc3 using `-O2 -std=c++17`. Each configuration used one warm-up run followed by five measured runs; the reported runtime is the median end-to-end wall-clock time.

For strong scaling, the process counts were $P \in \{1, 2, 4, 8\}$. Speedup and efficiency are

$$S(P) = \frac{T_1}{T_P}, \qquad E(P) = \frac{S(P)}{P}, \tag{1}$$

where T_1 is the runtime of the MPI implementation with one process. Communication profiling was performed separately on the largest input using MPI wrappers. "MPI-call time" includes communication as well as synchronization/waiting; "outside-MPI time" contains local computation and other non-MPI work.

2 Q3: Bitonic Sort

2.1 Correctness and implementation

Given the input constraints, the implementation first broadcasts N and explicitly rejects inputs unless both N and P are powers of two and N is divisible by P.Each rank then receives exactly $m = N/P$ elements using `MPI_Scatter` and sorts its local block using `std::sort`.

The process-level bitonic network is implemented by the nested (k, j) loops. At each comparator, rank r exchanges its sorted block with rank $r \oplus j$ using `MPI_Sendrecv`. Depending on the bitonic stage and rank direction, it keeps either the lower or upper m values using `take_low` or `take_high`. These compare-split operations preserve the local sorted-order invariant. Because P is a power of two, the sequence of XOR partners is the standard bitonic sorting network; after the final stage, the rank-ordered blocks are globally nondecreasing. `MPI_Gather` therefore reconstructs the fully sorted sequence at rank 0.

2.2 Performance results

Table 1: Q3 median runtime in seconds.

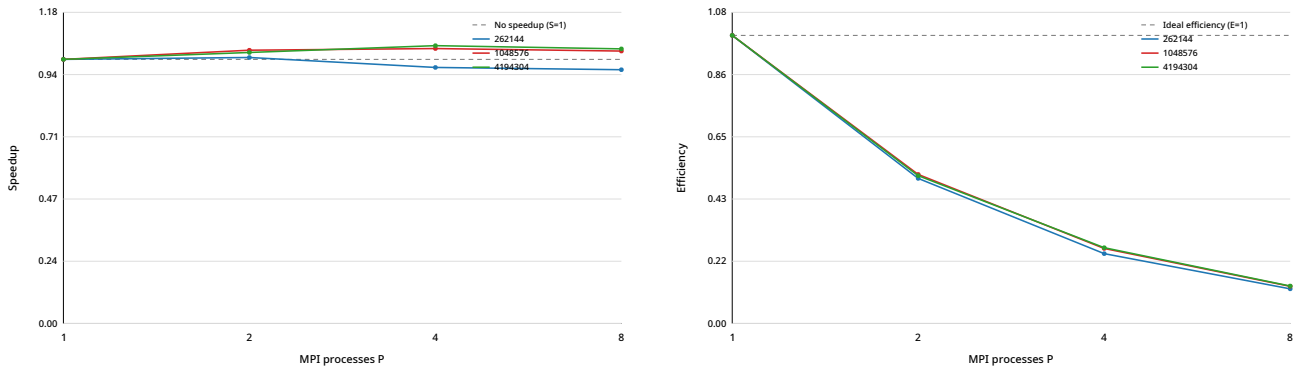
Input size	$P = 1$	$P = 2$	$P = 4$	$P = 8$
Small (N = 262,144)	0.302701	0.300489	0.312233	0.315031
Medium (N = 1,048,576)	0.948316	0.916297	0.910929	0.919365
Large (N = 4,194,304)	3.436922	3.348640	3.267161	3.305272

Table 2: Q3 speedup $S(P) = T_1/T_P$.

Input size	$P = 1$	$P = 2$	$P = 4$	$P = 8$
Small	1.000	1.007	0.969	0.961
Medium	1.000	1.035	1.041	1.031
Large	1.000	1.026	1.052	1.040

Table 3: Q3 parallel efficiency $E(P) = S(P)/P$.

Input size	$P = 1$	$P = 2$	$P = 4$	$P = 8$
Small	1.000	0.504	0.242	0.120
Medium	1.000	0.517	0.260	0.129
Large	1.000	0.513	0.263	0.130



(a) Speedup.

(b) Efficiency.

Figure 1: Q3 scaling with MPI process count.

Table 4: Q3 communication/computation profile for $N = 4{,}194{,}304$.

P	MPI-call time (s)	Outside-MPI time (s)	MPI fraction	Outside fraction
1	0.0059	3.3133	0.2%	99.8%
2	1.4264	1.7549	47.4%	52.6%
4	2.0493	0.8410	73.0%	27.0%
8	2.4204	0.4182	86.3%	13.7%

The largest input obtains the best observed speedup, about 1.052× at $P = 4$. Increasing the process count reduces the local sorting/comparison work, but the fraction of time spent inside MPI calls rises sharply. At $P = 8$, MPI calls account for about 86% of mean rank time.

3 Q6: Connected Components

3.1 Correctness and implementation

Rank 0 reads the adjacency lists. The code partitions the V vertices using

$$\texttt{counts[p]} = \lfloor V/P \rfloor + \mathbf{1}[p < V \bmod P],$$

so non-divisible values of V are handled explicitly. Each assigned adjacency list is packed and distributed with `MPI_Scatterv`. Every vertex is initialized with its own ID as its label.

Let $\ell_t(v)$ denote the label of vertex v after t completed iterations. Initially, $\ell_0(v) = v$. In one iteration the code computes

$$\ell_{t+1}(v) = \min(\ell_t(v), \{\ell_t(u) : u \in N(v)\}) .$$

By induction, $\ell_t(v)$ is the minimum vertex ID among vertices at graph distance at most t from v. For a valid undirected input graph, once t reaches the diameter of v's connected component, $\ell_t(v)$ is exactly the minimum vertex ID in that component. `MPI_Allgatherv` makes every iteration synchronous by reconstructing the complete label vector on every rank, and `MPI_Allreduce` terminates only when no rank reports a changed label. Rank 0 finally prints vertices in the loop order $0, \ldots, V-1$, so the required output ordering is also satisfied.

Empirical correctness was checked against the separate sequential DFS implementation.

3.2 Performance results

Table 5: Q6 median runtime in seconds.

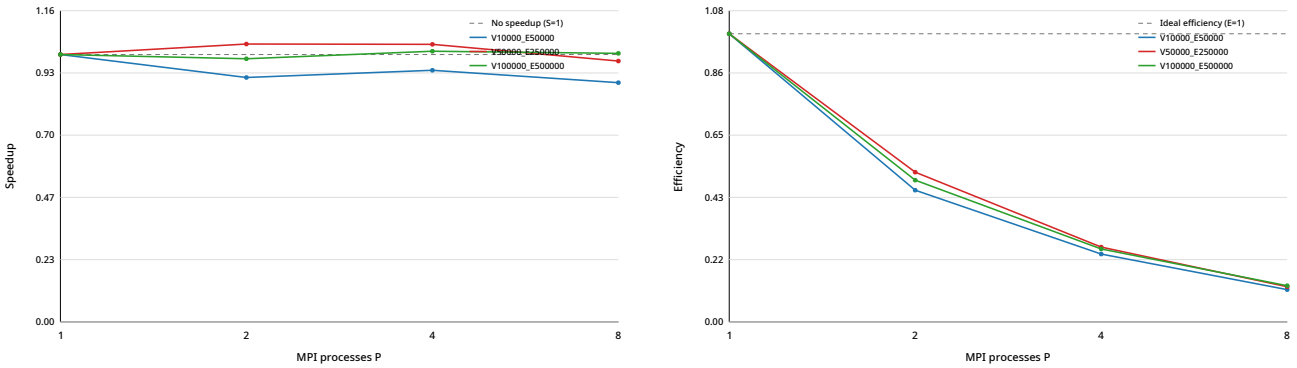
Input size	$P = 1$	$P = 2$	$P = 4$	$P = 8$
Small ($V = 10{,}000$, $E = 50{,}000$)	0.160387	0.175500	0.170441	0.179285
Medium ($V = 50{,}000$, $E = 250{,}000$)	0.461660	0.444316	0.444781	0.473209
Large ($V = 100{,}000$, $E = 500{,}000$)	0.808081	0.821265	0.798353	0.804701

Table 6: Q6 speedup $S(P) = T_1/T_P$.

Input size	$P = 1$	$P = 2$	$P = 4$	$P = 8$
Small	1.000	0.914	0.941	0.895
Medium	1.000	1.039	1.038	0.976
Large	1.000	0.984	1.012	1.004

Table 7: Q6 parallel efficiency $E(P) = S(P)/P$.

Input size	$P = 1$	$P = 2$	$P = 4$	$P = 8$
Small	1.000	0.457	0.235	0.112
Medium	1.000	0.520	0.259	0.122
Large	1.000	0.492	0.253	0.126



(a) Speedup.

(b) Efficiency.

Figure 2: Q6 scaling with MPI process count.

Table 8: Q6 communication/computation profile for $V = 100{,}000$, $E = 500{,}000$.

P	MPI-call time (s)	Outside-MPI time (s)	MPI fraction	Outside fraction
1	0.0005	0.6802	0.1%	99.9%
2	0.3106	0.3844	49.4%	50.6%
4	0.4380	0.1745	74.6%	25.4%
8	0.5158	0.0893	87.2%	12.8%

The performance is essentially flat over $P = 1$–8. For the largest graph the best measured value is $1.012\times$ at $P = 4$. The local vertex-processing work decreases with P, but communication cost for label exchange and synchronization become dominant.

4 Q7: Large-Scale Server Log Analytics

4.1 Correctness, data generation, and implementation

For the standard-input path used by the correctness harness, rank 0 reads records in chunks of at most 10^6 rows. The code computes disjoint per-rank row counts and offsets and distributes the integer fields and response times using two `MPI_Scatterv` calls. Thus every input row in a chunk is assigned to exactly one rank.

Each rank applies the same local accumulation routine to its records: a request is counted as successful iff `status_code < 400`; response-time sum/minimum/maximum, total bytes, and 2xx–5xx counts are updated directly; server counts and response-time sums are accumulated by server; endpoint counts/bytes and 60-second interval counts are accumulated in maps. The global result is then formed as follows:

- additive counters and server statistics use `MPI_Reduce` with `MPI_SUM`;
- response-time minimum and maximum use `MPI_MIN` and `MPI_MAX`;
- endpoint and interval maps are flattened, gathered to rank 0, and summed by key;
- server and endpoint rows are sorted by decreasing request count and then increasing ID, exactly matching the required Top-K tie rule;
- the busiest interval is computed from `timestamp/60`; on an otherwise unspecified tie the implementation deterministically chooses the smaller interval ID.

The reproducible performance generator uses seed $12345+N$, $K = 5$ and $S = 64$, with $N \in \{100{,}000, 500{,}000, 1{,}000{,}000\}$.

4.2 Performance results

Table 9: Q7 MPI median runtime in seconds.

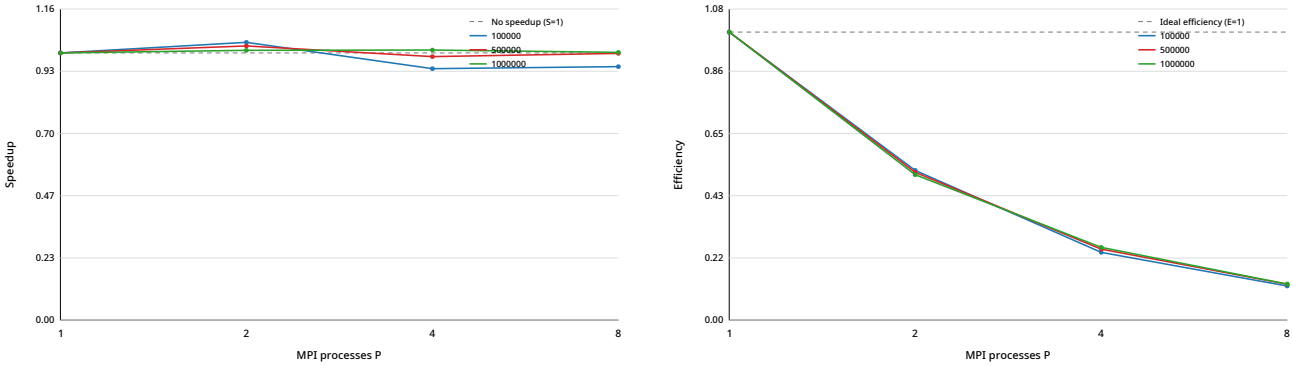
Input size	$P = 1$	$P = 2$	$P = 4$	$P = 8$
Small ($N = 100{,}000$)	0.164009	0.157778	0.174296	0.172863
Medium ($N = 500{,}000$)	0.374197	0.364693	0.379463	0.374869
Large ($N = 1{,}000{,}000$)	0.617441	0.611537	0.610742	0.616104

Table 10: Q7 MPI speedup $S(P) = T_{\mathrm{MPI},1}/T_{\mathrm{MPI},P}$.

Input size	$P = 1$	$P = 2$	$P = 4$	$P = 8$
Small	1.000	1.039	0.941	0.949
Medium	1.000	1.026	0.986	0.998
Large	1.000	1.010	1.011	1.002

Table 11: Q7 MPI parallel efficiency $E(P) = S(P)/P$.

Input size	$P = 1$	$P = 2$	$P = 4$	$P = 8$
Small	1.000	0.520	0.235	0.119
Medium	1.000	0.513	0.247	0.125
Large	1.000	0.505	0.253	0.125



(a) Speedup.

(b) Efficiency.

Figure 3: Q7 MPI strong scaling.

Table 12: Q7 sequential runtime versus the best observed MPI runtime.

N	Sequential (s)	Best MPI (s)	Best P	MPI/Sequential
100,000	0.051290	0.157778	2	3.076
500,000	0.248740	0.364693	2	1.466
1,000,000	0.487134	0.610742	4	1.254

Table 13: Q7 communication/computation profile for $N = 1{,}000{,}000$.

P	MPI-call time (s)	Outside-MPI time (s)	MPI fraction	Outside fraction
1	0.0070	0.5059	1.4%	98.6%
2	0.2386	0.2537	49.1%	50.9%
4	0.3771	0.1287	74.2%	25.8%
8	0.4387	0.0661	87.0%	13.0%

The MPI implementation shows little strong scaling for the tested sizes. The separate sequential implementation remains faster, but the MPI slowdown decreases from about 3.08× at $N = 100{,}000$ to about 1.25× at $N = 1{,}000{,}000$, showing that fixed MPI overhead is decreasing as the workload grows. Profiling also shows that synchronization/communication dominates mean rank time at larger process counts.

5 Conclusion

On this single-node setup, the tested workloads exhibit limited strong scaling. Q3 benefits modestly on its largest input and reaches a maximum observed speedup of about 1.05× at $P = 4$, while Q6 and Q7 remain close to 1×. Across all three programs, increasing P reduces local work but substantially increases the fraction of rank time spent inside MPI calls, which limits parallel efficiency. Q7 additionally shows that although MPI is slower than the sequential implementation at the tested sizes, its relative overhead decreases as the input becomes larger.